



Universidad Autónoma de Sinaloa

Licenciatura en Ingeniería de Software



Tarea 11: orquestación de microservicios, alta disponibilidad y túneles de gestión seguros

Materia:

Administración de Sistemas

Nombre del Profesor:

Dr. Herman Geovany Ayala Zúñiga

Nombre del alumno:

Espiritu Manzanarez Diego Manuel

Fecha:

08/05/2026

➤ Introducción y Arquitectura o lo que trate cada práctica

- Historial de Cambios:

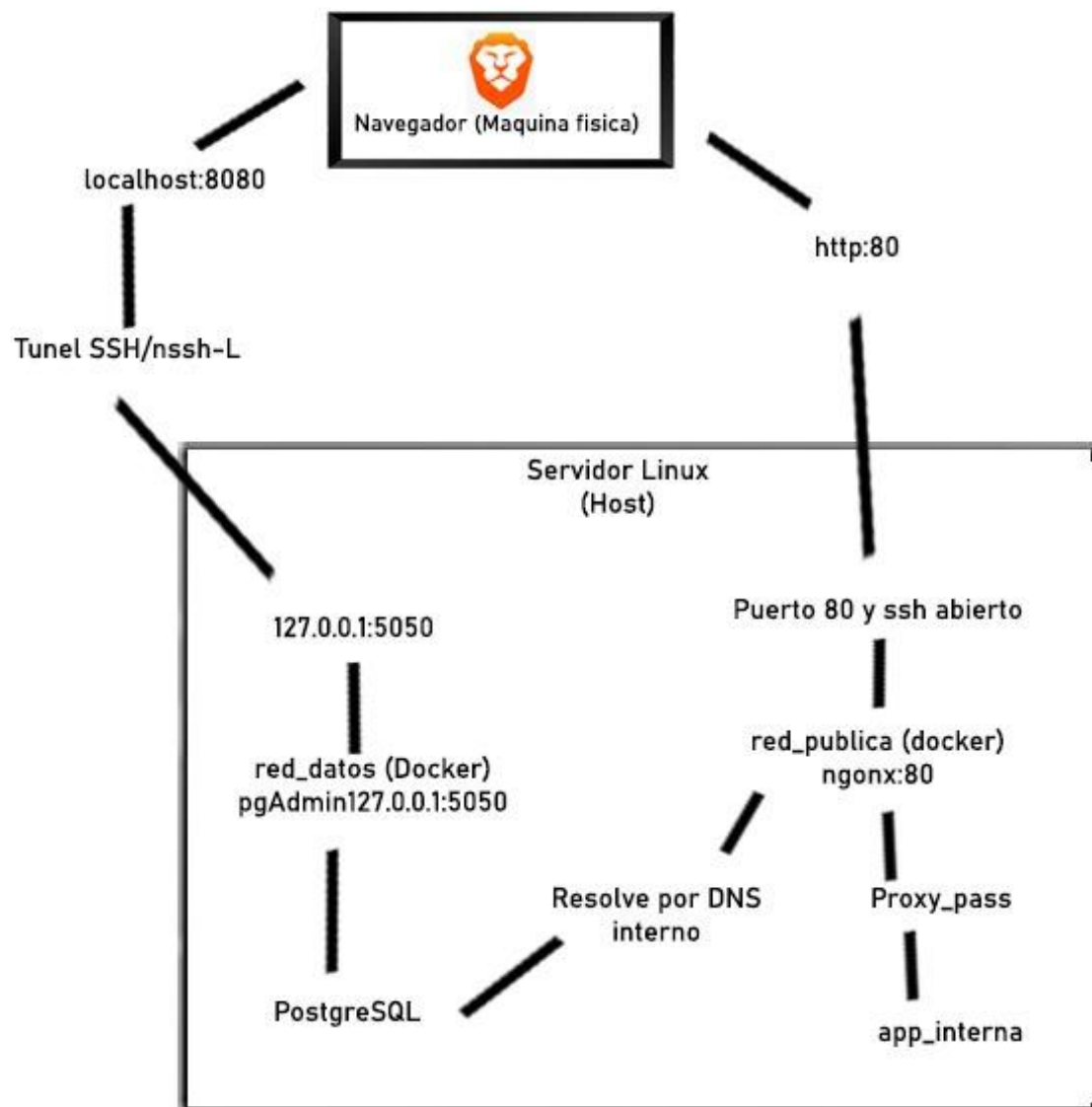
Repositorio: <https://github.com/Diegoesp-0/Administracion-de-Sistemas>

Versión	Fecha	Descripción del Cambio	Relación con Git
11.0	08/05/2026	Se creo la primera versión de la practica.	Tarea 11: Primera version
11.1	08/05/2026	Se corrigieron problemas del gestor de paquetes que no instalaba correctamente.	Tarea 11: Arreglos para instalación
11.2	08/05/2026	Se solucionaron problemas con zypper en la instalación.	Tarea 11: Problemas de zypper
11.3	08/05/2026	Se solucionaron problemas de interfaz.	Tarea 11: Mas arreglos
11.4	08/05/2026	Se soluciono el error que nginx no levantaba su servicio.	Tarea 11: Problemas de nginx
11.5	08/05/2026	Se termino de arreglar cosas como la interfaz e incompatibilidades.	Arreglos finales

- **Objetivo:**

Desplegar una infraestructura de microservicios coordinada mediante Docker Compose, aplicando principios de IaC: seguridad perimetral, separación de capas de red, persistencia de datos y administración cifrada vía túnel SSH.

- **Diagrama de Topología:**



➤ Guía de Uso de los Scripts

- Requisitos previos:

- Tener openSUSE con acceso a internet y permisos sudo.
- Tener un adaptador puente enp0s9.
- Usuario con permisos sudo.

➤ Instrucciones de Ejecución:

- 1- Ejecuta el comando “bash practica11.sh -i” para después poner el comando “newgrp docker”

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts> bash practica11.sh -i

==== Tarea 11 - Instalacion completa ====

==== Verificando dependencias ====

[INFO] Verificando instalacion de Docker...
[INFO] Docker no encontrado, instalando...
[sudo] contraseña para Espiritu:
[OK] Docker instalado correctamente
[INFO] Verificando servicio Docker...
[INFO] Iniciando servicio Docker...
[OK] Servicio Docker iniciado
[INFO] Verificando grupo docker...
[INFO] Usuario agregado al grupo docker
[AVISO] Ejecuta: newgrp docker y vuelve a correr el script
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts> newgrp docker
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts>
```

- 2- Despues ejecutar nuevamente el comando “bash practica11.sh -i”

```
==== Instalacion finalizada ====

==== Estado del stack ====

NAME          IMAGE                COMMAND                SERVICE    CREATED        STATUS              PORT
TS
t11_app       httpd:alpine         "httpd-foreground"    app_interna 26 seconds ago  Up 23 seconds      80/
tcp
t11_db        postgres:16-alpine   "docker-entrypoint.s..." postgresql 26 seconds ago  Up 23 seconds (healthy) 543
2/tcp
t11_nginx     nginx:alpine         "/docker-entrypoint..." nginx      24 seconds ago  Up 21 seconds      0.0
.0.0:80->80/tcp, [::]:80->80/tcp
t11_pgadmin   dpkg/pgadmin4        "/entrypoint.sh"      pgadmin    24 seconds ago  Up 1 second        443
/tcp, 127.0.0.1:5050->80/tcp

[INFO] Redes activas:

[INFO] Volumen de datos:
local      t11_db_data

[INFO] Para acceder a pgAdmin via tunel SSH:
ssh -L 8080:localhost:5050 Espiritu@192.168.56.112
Luego abrir: http://localhost:8080
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts>
```

3- Probar en la maquina física si hace ping.

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\diego>curl -m 3 http://192.168.56.112:5050
curl: (28) Connection timed out after 3002 milliseconds

C:\Users\diego>curl -m 3 http://192.168.56.112:5432
curl: (28) Connection timed out after 3001 milliseconds

C:\Users\diego>
```

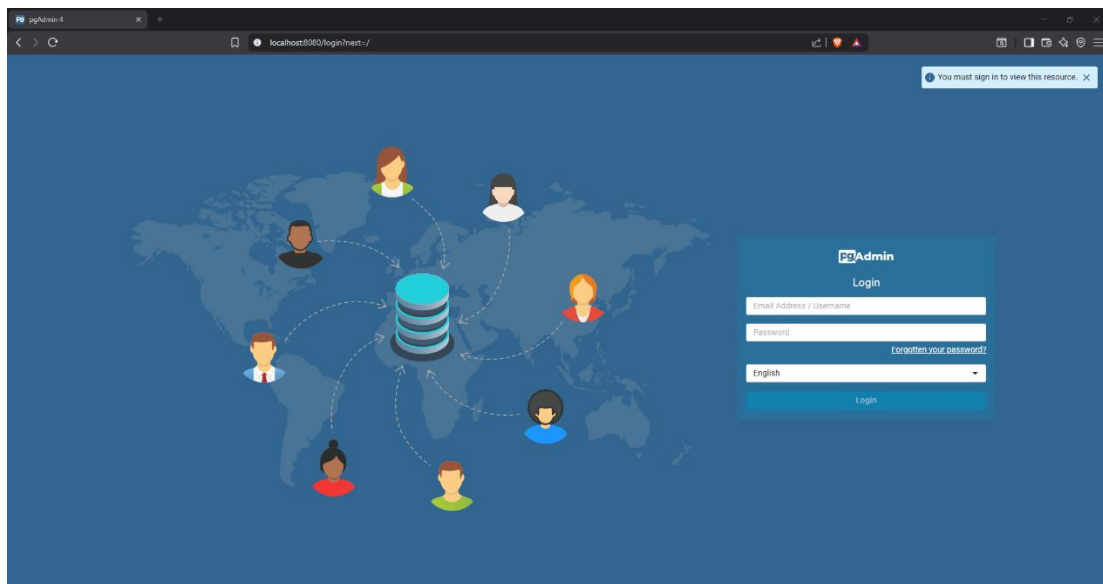
4- Verificar el ping en maquina física

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts> docker exec t11_nginx ping -c 2 postgresql
PING postgresql (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.135 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.103 ms

--- postgresql ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.103/0.119/0.135 ms
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts>
```

5- Hacer ssh en la maquina física de la siguiente forma e ir al navegador y poner "http://localhost:8080"

```
C:\Users\diego>ssh -L 8080:localhost:5050 Espiritu@192.168.56.112
(Espiritu@192.168.56.112) Password:
Have a lot of fun...
Espiritu@localhost:~>
```



➤ Bitácora de Desarrollo y Configuración:

- Explicación de Script:

El sistema está construido de forma modular donde cada archivo tiene una responsabilidad específica. `practica11.sh` actúa como punto de entrada único, carga todos los módulos con `source` y expone una interfaz de flags al administrador. Al arrancar detecta automáticamente qué versión de Docker Compose está disponible en el sistema y la guarda en `COMPOSE_CMD` para que todos los módulos la usen de forma consistente.

`colores.sh` es una utilidad de presentación que define funciones de impresión categorizadas por color según la naturaleza del mensaje: errores en rojo, confirmaciones en verde, información en amarillo y títulos en azul.

`docker_install.sh` y `T11_verificaciones.sh` trabajan en conjunto para garantizar que el entorno esté listo antes de cualquier operación: verifican Docker, su servicio, el grupo del usuario y Docker Compose, con múltiples niveles de fallback que van desde `zypper` hasta descarga directa desde GitHub.

`T11_infraestructura.sh` genera todos los archivos que definen el entorno: el `.env` con permisos 600, el `nginx.conf` con `server_tokens off` para ocultar la versión del servidor, la página de la aplicación interna y el `docker-compose.yml` con los cuatro contenedores, las dos redes bridge y el volumen nombrado. Todas las funciones verifican si el archivo ya existe antes de crearlo, haciendo el script idempotente. `T11_firewall.sh` aplica una política de defensa en profundidad cerrando los puertos 5050 y 5432 a nivel de firewall, complementando el aislamiento que ya provee Docker.

`T11_stack.sh` maneja el ciclo de vida del stack e incluye un bucle de espera activa que consulta el healthcheck de PostgreSQL cada 5 segundos, garantizando que el script no termine hasta que la infraestructura esté completamente lista. `T11_pruebas.sh` implementa el protocolo de validación automatizando las cuatro pruebas de aceptación: verifica el aislamiento de puertos con `ss` y `firewall-cmd`, valida el DNS interno ejecutando un ping desde dentro del contenedor `nginx`, muestra el comando del túnel SSH con la IP detectada dinámicamente, y prueba la persistencia insertando datos, bajando el stack y confirmando que los registros sobreviven al reinicio.

- Evidencias de Configuración:

1- Archivo .env

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts> cat ~/Admin
# Base de datos PostgreSQL
DB_USER=admin
DB_PASS=cambiar_en_produccion
DB_NAME=appdb

# pgAdmin
PGADMIN_EMAIL=admin@local.com
PGADMIN_PASS=cambiar_en_produccion

# Puerto local para pgAdmin (solo accesible desde localhost via tunel SSH)
PGADMIN_PORT=5050
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts>
```

2- Docker-compose

```
services:

  nginx:
    image: nginx:alpine
    container_name: t11_nginx
    ports:
      - "80:80"
    volumes:
      - /opt/t11/nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - red_publica
      - red_datos
    restart: always
    depends_on:
      - app_interna

  app_interna:
    image: httpd:alpine
    container_name: t11_app
    volumes:
      - /opt/t11/html:/usr/local/apache2/htdocs:ro
    networks:
      - red_publica
    restart: always

  postgresql:
    image: postgres:16-alpine
    container_name: t11_db
    environment:
```

3- Nginx.conf

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts> cat /opt/t11/nginx.conf
user nginx;
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    server_tokens off;

    upstream app_backend {
        server app_interna:80;
    }

    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://app_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_hide_header X-Powered-By;
        }
    }
}
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-11/Scripts>
```


➤ Conclusiones y Referencias

- Lecciones Aprendidas:

Aprendí que en entornos reales los servicios no viven solos, dependen unos de otros y hay que planificar eso. También que las contraseñas no van escritas directo en los archivos de configuración.

- Bibliografía:

<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-compose-on-ubuntu-20-04-es>

<https://openwebinars.net/blog/que-es-docker/>

<https://docs.docker.com/get-started/overview/>